Week 2 Project: Advanced Line Calculations

Thermal Analysis, Voltage Drop, and Conductor Sizing

Enrique Antonio Raudales Rodriguez

October 26, 2025

University of Almería
Medium and Low Voltage Electrical Installations
Academic Year 2025-2026

APPENDIX A

A References

1. **Reglamento Electrotécnico para Baja Tensión (REBT)**, Real Decreto 842/2002. Specifically:
   - **ITC-BT-06:** Redes aéreas para distribución en baja tensión.
   - **ITC-BT-07:** Redes subterráneas para distribución en baja tensión.
   - **ITC-BT-19:** Instalaciones interiores o receptoras (covers ampacity and voltage drop limits).
2. **IEC 60287-1-1**, "Electric cables - Calculation of the current rating".
3. **ELEK Software**. (2023). *Skin and Proximity Effects on AC Resistance Calculations.*
4. **K Malmedal C Bates D Cain**. (2023). *the measurement of soil thermal stability thermal resistivity and underground cable ampacity.*
5. IEEE Std 738-2023, *IEEE Standard for Calculating the Current-Temperature Relationship of Bare Overhead Conductors*, IEEE Power and Energy Society, 2023.
6. ScienceDirect Topics, *Prandtl Number*. Accessed Oct 26, 2025. [Online]. Available: `https://www.sciencedirect.com/topics/engineering/prandtl-number`
7. Plota, A.; Masek, A. Lifetime Prediction Methods for Degradable Polymeric Materials A Short Review. *Materials* **2020**, 13, 4507.
8. Table A-9, Properties of air at 1 atm pressure. (Source likely a Thermodynamics or Heat Transfer textbook, e.g., Cengel & Ghajar, *Heat and Mass Transfer: Fundamentals & Applications*).
9. ROHINI COLLEGE OF ENGINEERING & TECHNOLOGY, *Kelvin's Law for Economic Conductor Selection.* (Cites V.K. Mehta, *Principles of Power System*).
10. IEC 60287-1-1:2006, *Electric cables - Calculation of the current rating - Part 1-1: Current rating equations (100 % load factor) and calculation of losses - General.* International Electrotechnical Commission, 2006.
11. Churchill, S. W.; Bernstein, M. A Correlating Equation for Forced Convection From Gases and Liquids to a Circular Cylinder in Crossflow. *J. Heat Transfer* **1977**, 99(2), 300–306.
12. Churchill, S. W.; Chu, H. H. S. Correlating equations for laminar and turbulent free convection from a vertical plate. *Int. J. Heat Mass Transfer* **1975**, 18(11), 1323–1329.
13. Internal Document: "Week 2 Report Outline/Code", Oct 2025.

APPENDIX B

B Tables and Assumed Values

B.1 List of Tables

List of Tables

B.2 Summary of Material Properties

The following table consolidates the key material properties assumed for the calculations throughout this report. These values are representative and based on standard engineering references.

Table 1: Consolidated Material Properties and Assumed Values

Material	Property	Value
Copper	Density (ρ)	$8960\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$385\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Conductivity (σ)	$56\,\mathrm{m\,\Omega^{-1}\,mm^{-2}}$
	Temp. Coefficient (α)	$0.003\,93\,\mathrm{K^{-1}}$
Aluminum	Density (ρ)	$2700\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$900\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Conductivity (σ)	$36\,\mathrm{m\,\Omega^{-1}\,mm^{-2}}$
	Temp. Coefficient (α)	$0.004\,03\,\mathrm{K^{-1}}$
ACSR	Density (approx. avg.)	$3980\,\mathrm{kg\,m^{-3}}$
	Specific Heat (approx. avg.)	$880\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Conductivity (σ)	$34\,\mathrm{m\,\Omega^{-1}\,mm^{-2}}$
PVC	Density (ρ)	$1400\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$1000\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Thermal Conductivity (k)	$0.17\,\mathrm{W\,m^{-1}\,K^{-1}}$
XLPE	Density (ρ)	$920\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$2300\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Thermal Conductivity (k)	$0.28\,\mathrm{W\,m^{-1}\,K^{-1}}$
Soil	Thermal Resistivity (ρ_{soil})	$1.5\,\mathrm{K\,m\,W^{-1}}$

C Formulas

C.1 AC Resistance at Temperature

$$R_{AC,T} = R_{DC,20} \cdot (1 + k_{\text{skin}} + k_{\text{prox}}) \cdot (1 + \alpha(T - T_{\text{ref}})) \tag{1}$$

C.2 Blondel's Formula (Three-Phase)

$$\Delta V = \sqrt{3} \cdot I \cdot (R \cos\varphi + X_L \sin\varphi) \tag{2}$$

C.3 Required Section (Voltage Drop)

$$s = \frac{\sqrt{3} \cdot L \cdot I \cdot \cos\varphi}{\sigma \cdot \Delta V_{\text{max}}} \tag{3}$$

C.4 Lifecycle Cost

$$C_{\text{total}} = C_{\text{initial}} + \sum (P_{\text{loss}} \cdot t_{\text{op}} \cdot \text{cost}_{\text{kWh}}) \tag{4}$$

APPENDIX D

D MATLAB Source Code

D.1 week2_master.m

```
%
    ========================================================================

% MASTER SCRIPT for Week 2: Advanced Line Calculations
%
    ========================================================================

% Description:
% This script is the central launcher for all modules developed for the
% Week 2 coursework (E, F, and G). It provides a menu to select which
% analysis suite to run.
%
    ========================================================================


%% --- Cleanup and Initialization ---
clc;
clear;
close all;

%% --- Main Menu Loop ---
while true
    % A custom menu function is assumed to exist, named centeredMenu2
    analysisChoice = centeredMenu2('Select Week 2 Analysis Suite:', ...
                                   'Problem E: Conductor Heating
    Analysis', ...
                                   'Problem F: Voltage Drop Analysis',
    ...
                                   'Problem G: Conductor Sizing Analysis
    ', ...
                                   'Exit Program');

    try
        switch analysisChoice
            case 1
                disp('--- Launching Conductor Heating Analysis (Module E
    ) ---');
                run('E0_Run_Analysis_and_Report.m');
                disp('Press any key to return to the main menu...');
                pause;

            case 2
                disp('--- Launching Voltage Drop Analysis (Module F) ---
    ');
                run('F1_VoltageDropAnalysis.m');
                disp('Press any key to return to the main menu...');
                pause;

            case 3
```

```
                disp('--- Launching Conductor Sizing Analysis (Module G) ---');
                G1_ConductorSizingAnalysis();
                disp('Press any key to return to the main menu...');
                pause;

            case 4
                disp('Exiting the Week 2 Analysis Suite.');
                return;

            case 0
                disp('Menu closed. Exiting the Week 2 Analysis Suite.');
                return;
        end

    catch ME
        fprintf('\nERROR encountered while running the selected module.\n');
        fprintf('MATLAB reported: "%s" in file "%s" at line %d.\n', ME.message, ME.stack(1).name, ME.stack(1).line);
        disp('Press any key to return to the main menu...');
        pause;
    end
end
```

Listing 1: Central launcher for all Week 2 modules.

D.2 E0_Run_Analysis_and_Report.m

```
%% ========================================================================
% FINAL SCRIPT for Conductor Heating Analysis and Reporting
%% ========================================================================
% Description:
% This script serves as the final entry point for the Conductor Heating
% module. It calls the main analysis function and then uses the
% returned data to display a comprehensive summary table.
%% ========================================================================


%% --- Run the Full Analysis ---
results = E1_ConductorHeatingAnalysis();


%% --- Generate and Display the Final Report Table ---
if isempty(results)
    disp('Analysis cancelled by user. No report generated.');
    return;
end

fprintf('\n\n\n
    ========================================================================\n');
```

```
fprintf('    THERMAL ANALYSIS AND MAXIMUM CURRENT CALCULATION REPORT\n');
fprintf('
    ==========================================================================\n\n');

fprintf('--- 1. Input Parameters ---\n');
fprintf('%-35s: %s\n', 'Scenario', results.scenario);
fprintf('%-35s: %s / %s\n', 'Conductor / Insulation', results.
    materialName, results.insulationName);
fprintf('%-35s: %.1f mm^2\n', 'Conductor Cross-Section', results.
    crossSection);

envFields = fieldnames(results.envParams);
fprintf('\n   Environment-Specific Parameters:\n');
for i = 1:length(envFields)
    if ~strcmp(envFields{i}, 'scenario')
        fprintf('   %-32s: %.2f\n', envFields{i}, results.envParams.(
    envFields{i}));
    end
end

fprintf('\n--- 2. Intermediate Calculated Values ---\n');
fprintf('%-35s: %.5f Ohm\n', 'DC Resistance at 20 C', results.R_20_DC);
fprintf('%-35s: %.5f Ohm\n', 'AC Resistance at 20 C', results.R_20_AC);
fprintf('%-35s: %.5f Ohm\n', 'AC Resistance at T_max', results.R_Tmat_AC
    );
fprintf('%-35s: %.2f W\n', 'Max Heat Dissipation', results.
    P_dissipated_max);

fprintf('\n--- 3. Final Analysis Results ---\n');
fprintf('
    --------------------------------------------------------------------------\
    n');
fprintf('%-35s: %.2f A\n', 'MAXIMUM ADMISSIBLE CURRENT (I_max)', results
    .I_max);
fprintf('
    --------------------------------------------------------------------------\
    n\n');
```

Listing 2: Main analysis runner for thermal analysis.

D.3 E1_ConductorHeatingAnalysis.m

```
function results = E1_ConductorHeatingAnalysis()
%
    ========================================================================

% MAIN FUNCTION for Conductor Thermal Analysis
% Returns a struct 'results' with all key values.
%
    ========================================================================

results = [];
T_ref = 20;
rho_den_aluminum = 2700; cp_aluminum = 900;
rho_den_copper = 8960; cp_copper = 385;
```

```
T_mat_xlpe = 90; T_mat_pvc = 70;
alpha_al = 0.00403; alpha_cu = 0.00393;
sigma_al = 36; sigma_cu = 56;

installationChoice = 1; % Assume Underground for report
switch installationChoice
    case 1 % Underground
        scenarioName = 'Underground'; materialName = 'Aluminum';
    insulationName = 'XLPE';
        crossSection = 150; insulatorThickness = 2.2;
        envParams.scenario = 'underground'; envParams.T_env = 20;
        envParams.rho_soil = 1.5; envParams.burial_depth = 0.8;
        density_cond = rho_den_aluminum; cp_cond = cp_aluminum;
        T_mat = T_mat_xlpe; alpha = alpha_al; sigma = sigma_al;
        envParams.k_insulator = 0.28;
    % Other cases omitted for brevity
end

lineLength = 1; frequency = 50;

r_inner_m = sqrt(crossSection / pi) / 1000;
diameter_m = 2 * r_inner_m;
r_outer_m = r_inner_m + (insulatorThickness / 1000);
R_20_DC = lineLength / (sigma * crossSection);
[k_skin, k_proximity] = E3b_AC_Resistance_Factor(frequency, diameter_m,
    sigma);
R_20_AC = R_20_DC * (1 + k_skin + k_proximity);
[I_max, R_Tmat_AC, ~, P_dissipated_max] = E2_MaximumCurrent(R_20_AC,
    alpha, T_mat, T_ref, envParams, lineLength, r_inner_m, r_outer_m);

results.scenario=scenarioName; results.materialName=materialName;
    results.insulationName=insulationName;
results.crossSection=crossSection; results.envParams=envParams;
results.R_20_DC=R_20_DC; results.R_20_AC=R_20_AC; results.R_Tmat_AC=
    R_Tmat_AC;
results.P_dissipated_max=P_dissipated_max; results.I_max=I_max;
end
```

Listing 3: Core function for thermal analysis.

D.4 E2_MaximumCurrent.m

```
function [I_max, R_Tmat, T_surface, P_diss_max] = E2_MaximumCurrent(R_20
    , alpha, T_mat, T_ref, envParams, lineLength, r_inner, r_outer)
R_Tmat = E3_ResistanceTemperatureCorrection(R_20, alpha, T_mat, T_ref);
options = optimset('Display','off');
if r_outer <= r_inner
    R_thermal_ins = inf;
else
    R_thermal_ins = log(r_outer/r_inner) / (2*pi*envParams.k_insulator*
    lineLength);
end
balance_eq = @(T_s) (T_mat - T_s)/R_thermal_ins - E7_HeatDissipation(T_s
    , envParams, r_outer, lineLength);
try
    T_surface = fzero(balance_eq, T_mat, options);
catch
```

```
    T_surface = T_mat;
end
P_diss_max = E7_HeatDissipation(T_surface, envParams, r_outer,
    lineLength);
if R_Tmat > 0
    I_max = sqrt(P_diss_max / R_Tmat);
else
    I_max = inf;
end
end
```

Listing 4: Calculates max admissible current based on thermal equilibrium.

D.5 E3_ResistanceTemperatureCorrection.m

```
function R_T = E3_ResistanceTemperatureCorrection(R_20, alpha, T, T_ref)
R_T = R_20 * (1 + alpha * (T - T_ref));
end
```

Listing 5: Corrects resistance for operating temperature.

D.6 E3b_AC_Resistance_Factor.m

```
function [k_skin, k_proximity] = E3b_AC_Resistance_Factor(freq, diameter
    , sigma)
cross_section_m2 = pi * (diameter/2)^2;
sigma_Sm = sigma * 1e6;
R_dc_per_meter = 1 / (sigma_Sm * cross_section_m2);
if R_dc_per_meter == 0, x_s_sq = 0;
else, x_s_sq = (8 * pi * freq / R_dc_per_meter) * 1e-7; end
k_skin = (x_s_sq^2) / (192 + 0.8 * x_s_sq^2);
k_proximity = 0.05; % Assumed typical value
end
```

Listing 6: Calculates skin and proximity effect factors.

D.7 E4_ThermalEquilibrium.m

```
function T_eq = E4_ThermalEquilibrium(I, R_20, alpha, T_ref, envParams,
    length, ~, r_outer)
heat_balance_error = @(T) (E3_ResistanceTemperatureCorrection(R_20,
    alpha, T, T_ref) * I^2) - (E7_HeatDissipation(T, envParams, r_outer,
    length));
try
    options = optimset('Display','off');
    T_eq = fzero(heat_balance_error, envParams.T_env, options);
catch
    T_eq = envParams.T_env;
end
end
```

Listing 7: Solves for the steady-state temperature of a conductor.

D.8 E5_HeatingCurves.m

```
function E5_HeatingCurves(I_max, T_mat, envParams, R_20_DC, R_20_AC,
    alpha, T_ref, lineLength, r_inner, r_outer, materialName,
    insulationName)
current_vector = linspace(0, I_max * 1.2, 100);
temp_vector_ac = zeros(size(current_vector));
for i = 1:length(current_vector)
    temp_vector_ac(i) = E4_ThermalEquilibrium(current_vector(i), R_20_AC
    , alpha, T_ref, envParams, lineLength, r_inner, r_outer);
end
figure; hold on;
plot(current_vector, temp_vector_ac, 'r-', 'LineWidth', 2, 'DisplayName'
    , 'Equilibrium Temp (AC)');
line([0, I_max * 1.2], [T_mat, T_mat], 'Color', 'g', 'LineStyle', '--',
    'LineWidth', 1.5, 'DisplayName', sprintf('Max Temp (%.0f C)', T_mat))
    ;
line([I_max, I_max], [envParams.T_env, T_mat], 'Color', 'k', 'LineStyle'
    , ':', 'LineWidth', 1.5, 'DisplayName', sprintf('Max AC Current (%.1f
     A)', I_max));
title(sprintf('Heating Curve for %s / %s', materialName, insulationName)
    );
xlabel('Current (A)'); ylabel('Equilibrium Temperature (C)');
legend('Location', 'northwest'); grid on; box on;
ylim([envParams.T_env - 10, T_mat + 20]);
hold off;
end
```

Listing 8: Generates plots of equilibrium temperature vs. current.

D.9 E6_TransientHeating.m

```
function [time, temp] = E6_TransientHeating(I, time_span, envParams,
    R_20, alpha, T_ref, length, ~, r_outer, mass, cp)
    function dTdt = heat_balance_ode(~, T)
        P_generated = E3_ResistanceTemperatureCorrection(R_20, alpha, T,
    T_ref) * I^2;
        P_dissipated = E7_HeatDissipation(T, envParams, r_outer, length)
    ;
        dTdt = (P_generated - P_dissipated) / (mass * cp);
    end
options = odeset('RelTol', 1e-4, 'AbsTol', 1e-6);
[time, temp] = ode45(@heat_balance_ode, time_span, envParams.T_env,
    options);
end
```

Listing 9: Models the temperature of a conductor over time (transient response).

D.10 E7_HeatDissipation.m

```
function P_diss = E7_HeatDissipation(T_conductor, envParams, r_outer,
    length)
if strcmp(envParams.scenario, 'underground')
    P_diss = dissipation_underground(T_conductor, envParams.T_env,
    envParams.rho_soil, envParams.burial_depth, r_outer, length);
```

```
else % overhead
    P_diss = dissipation_overhead(T_conductor, envParams.T_env,
    envParams.wind_speed, envParams.solar_irradiance, envParams.
    emissivity, envParams.absorptivity, r_outer, length);
end
end
function P_cond = dissipation_underground(T_conductor, T_soil, rho_soil,
    H, r_outer, L)
    if r_outer > 0 && H > r_outer
        R_thermal_per_meter = (rho_soil / (2 * pi)) * acosh(H / r_outer)
    ;
        R_thermal_total = R_thermal_per_meter / L;
        P_cond = (T_conductor - T_soil) / R_thermal_total;
    else, P_cond = 0; end
end
function P_net = dissipation_overhead(T_conductor, T_air, V_wind,
    Q_solar, epsilon, alpha_solar, r_outer, L)
    P_conv = calculate_convection(T_conductor, T_air, V_wind, r_outer *
    2, L);
    P_rad = calculate_radiation(T_conductor, T_air, epsilon, r_outer *
    2, L);
    P_solar_gain = calculate_solar_gain(Q_solar, alpha_solar, r_outer *
    2, L);
    P_net = (P_conv + P_rad) - P_solar_gain;
end
function P_conv = calculate_convection(T_conductor, T_air, V_wind,
    D_outer, L)
    k_air = 0.026;
    if V_wind > 0, Re = V_wind*D_outer/1.5e-5; Nu = 0.3+(0.62*Re
    ^0.5*0.71^0.33)/(1+(0.4/0.71)^0.66)^0.25;
    else, Gr = 9.81*(1/(T_air+273.15))*abs(T_conductor-T_air)*D_outer
    ^3/(1.5e-5)^2; Nu = (0.6+(0.387*(Gr*0.71)^0.166)/(1+(0.559/0.71)
    ^0.562)^0.296)^2; end
    h = (k_air/D_outer)*Nu; surface_area = pi*D_outer*L; P_conv = h*
    surface_area*(T_conductor-T_air);
end
function P_rad = calculate_radiation(T_conductor, T_air, epsilon,
    D_outer, L)
    sigma_boltz=5.67e-8; surface_area=pi*D_outer*L; T_cond_K=T_conductor
    +273.15; T_air_K=T_air+273.15;
    P_rad = sigma_boltz*epsilon*surface_area*(T_cond_K^4-T_air_K^4);
end
function P_solar = calculate_solar_gain(Q_solar, alpha_solar, D_outer, L
    )
    projected_area = D_outer * L; P_solar = Q_solar * alpha_solar *
    projected_area;
end
```

Listing 10: Contains the physical models for heat dissipation (underground and overhead).

D.11 E8_GroupingFactor.m

```
function k_g = E8_GroupingFactor(num_cables)
if num_cables <= 3, k_g = 1.0; elseif num_cables <= 6, k_g = 0.8; else,
    k_g = 0.7; end
end
```

Listing 11: Provides a derating factor for grouped cables.

D.12 F1_VoltageDropAnalysis.m

```
function F1_VoltageDropAnalysis()
    installationSegments = {
        struct('name', 'LGA', 'length', 15, 'loadCurrent', 50, 'cos_phi', 0.9, 'conductor_s', 25),
        struct('name', 'DI', 'length', 20, 'loadCurrent', 20, 'cos_phi', 0.85, 'conductor_s', 10),
        struct('name', 'Internal', 'length', 12, 'loadCurrent', 16, 'cos_phi', 0.95, 'conductor_s', 4)
    };
    U_source = 230; lineType = 'single-phase'; sigma = 56; freq = 50; r_m = 0.08e-3;
    voltage_in = U_source; results = cell(size(installationSegments));
    for i = 1:length(installationSegments)
        seg = installationSegments{i};
        dia_m = 2*sqrt(seg.conductor_s/pi)/1000;
        [ks, kp] = E3b_AC_Resistance_Factor(freq, dia_m, sigma);
        r_ac = (1/(sigma*seg.conductor_s))*(1+ks+kp);
        seg.voltageDrop = F2_calculateExactVoltageDrop(lineType, seg.length, seg.loadCurrent, r_ac, r_m, seg.cos_phi);
        seg.voltage_out = voltage_in - seg.voltageDrop;
        results{i} = seg; voltage_in = seg.voltage_out;
    end
    F4b_checkInstallationCompliance(results, U_source);
end
```

Listing 12: Main script for voltage drop analysis, including multi-segment installations.

D.13 F2_calculateExactVoltageDrop.m

```
function deltaU = F2_calculateExactVoltageDrop(lineType, d, I, r, xL, cos_phi)
sin_phi = sqrt(1 - cos_phi^2);
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else, phase_factor = 2; end
deltaU = phase_factor * I * (r*d*cos_phi + xL*d*sin_phi);
end
```

Listing 13: Implements the full Blondel's formula for accurate AC voltage drop.

D.14 F3_calculateSimplifiedVoltageDrop.m

```
function deltaU_simple = F3_calculateSimplifiedVoltageDrop(lineType, d, I, r, cos_phi)
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else, phase_factor = 2; end
deltaU_simple = phase_factor * d * I * r * cos_phi;
end
```

Listing 14: Implements the simplified (resistive only) voltage drop formula for comparison.

D.15 F4_checkREBTCompliance.m

```
function [isCompliant, percentDrop, limit] = F4_checkREBTCompliance(
    deltaU, U_source, circuitType)
if strcmp(circuitType, 'lighting'), limit = 3.0; else, limit = 5.0; end
percentDrop = (deltaU / U_source) * 100;
isCompliant = percentDrop <= limit;
end
```

Listing 15: Checks if a single line's voltage drop complies with REBT limits.

D.16 F4b_checkInstallationCompliance.m

```
function F4b_checkInstallationCompliance(results, U_source)
fprintf('\n--- REBT COMPLIANCE CHECK ---\n');
total_drop = U_source - results{end}.voltage_out;
fprintf('Total Voltage Drop: %.2f V (%.2f%%)\n', total_drop, (total_drop
    /U_source)*100);
end
```

Listing 16: Checks a multi-segment installation against tiered REBT limits.

D.17 F5_plotVoltageProfile.m

```
function [] = F5_plotVoltageProfile(U_source, deltaU_total_ac,
    total_length, circuitType)
length_vector = linspace(0, total_length, 200);
voltage_profile_ac = U_source - (deltaU_total_ac / total_length) *
    length_vector;
if strcmp(circuitType, 'lighting'), limit_percent = 4.5/100; else,
    limit_percent = 6.5/100; end
min_voltage_limit = U_source * (1 - limit_percent);
figure; hold on;
plot(length_vector, voltage_profile_ac, 'r-', 'LineWidth', 2, '
    DisplayName', 'Voltage Profile (AC Res.)');
line([0, total_length], [min_voltage_limit, min_voltage_limit], 'Color',
     'g', 'LineStyle', '--', 'DisplayName', sprintf('REBT Limit (%.2f V)'
    , min_voltage_limit));
title('Voltage Profile Along the Line'); xlabel('Distance from Source (m
    )'); ylabel('Voltage (V)');
grid on; box on; legend('Location', 'southwest');
ylim([min_voltage_limit - (0.02*U_source), U_source * 1.02]);
hold off;
end
```

Listing 17: Plots the voltage profile for a single line.

D.18 F6_plotInstallationProfile.m

```
function F6_plotInstallationProfile(results, U_source)
numSegments = length(results);
node_voltages = [U_source, cellfun(@(x) x.voltage_out, results)'];
node_lengths = [0, cumsum(cellfun(@(x) x.length, results))'];
segment_names = cellfun(@(x) x.name, results, 'UniformOutput', false);
figure; hold on;
stairs(node_lengths, node_voltages, 'b-', 'LineWidth', 2, 'DisplayName',
     'Voltage Profile');
```

```
plot(node_lengths, node_voltages, 'ro', 'MarkerFaceColor','r', '
    HandleVisibility','off');
limit_total_percent = 5.0 / 100; % Simplified for example
min_voltage_limit = U_source * (1 - limit_total_percent);
line([0, node_lengths(end)], [min_voltage_limit, min_voltage_limit], '
    Color', 'k', 'LineStyle', '--', 'DisplayName', sprintf('Overall REBT
    Limit (%.2f V)', min_voltage_limit));
title('Voltage Profile Across Complete Installation');
xlabel('Cumulative Line Length (m)'); ylabel('Voltage (V)');
grid on; box on; legend('show', 'Location', 'southwest');
for i = 1:numSegments
    x_pos = node_lengths(i) + (node_lengths(i+1) - node_lengths(i))/2;
    y_pos = node_voltages(i+1) + 5;
    text(x_pos, y_pos, strrep(segment_names{i}, '_', ' '), '
    HorizontalAlignment', 'center', 'BackgroundColor', 'w');
end
hold off;
end
```

Listing 18: Plots the voltage profile across a multi-segment installation.

D.19 G1_ConductorSizingAnalysis.m

```
function G1_ConductorSizingAnalysis()
sigma_aluminum = 36; LCC_years = 20; hours_per_year = 4000; cost_per_kWh
    = 0.15;
U_source = 400; lineLength = 200; loadCurrent = 150; cos_phi = 0.85;
material = 'Aluminum';
[~, ~, sections_data] = G3_selectStandardSection(0, material);
s_required_vd = G2_calculateRequiredSection('three-phase', lineLength,
    loadCurrent, cos_phi, sigma_aluminum, U_source * 0.05);
s_technical_data = G3_selectStandardSection(s_required_vd, material);
s_technical = s_technical_data.section;
sections_to_analyze = sections_data(arrayfun(@(x) x.section,
    sections_data) >= s_technical);
num_sections = numel(sections_to_analyze);
total_costs=zeros(1,num_sections); cable_costs=zeros(1,num_sections);
    loss_costs=zeros(1,num_sections);
for i = 1:num_sections
    [total_costs(i), cable_costs(i), loss_costs(i)] =
    G4_calculateLifecycleCost(sections_to_analyze(i), 'three-phase',
    lineLength, loadCurrent, sigma_aluminum, LCC_years, hours_per_year,
    cost_per_kWh);
end
[~, optimal_idx] = min(total_costs);
s_optimal = sections_to_analyze(optimal_idx).section;
G5_plotEconomicAnalysis(sections_to_analyze, cable_costs, loss_costs,
    total_costs, s_technical, s_optimal);
fprintf('--- SIZING RESULTS ---\n');
fprintf('Technically Compliant Section: %d mm^2\n', s_technical);
fprintf('Economically Optimal Section: %d mm^2\n', s_optimal);
end
```

Listing 19: Main script for conductor sizing and economic analysis.

D.20 G2_calculateRequiredSection.m

```
function s = G2_calculateRequiredSection(lineType, d, I, cos_phi, sigma,
    deltaU_max)
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else,
    phase_factor = 2; end
s = (phase_factor * d * I * cos_phi) / (sigma * deltaU_max);
end
```

Listing 20: Calculates the minimum required cross-section based on voltage drop.

D.21 G3_selectStandardSection.m

```
function [selected_section, ~, standard_sections_data] =
    G3_selectStandardSection(s_required, material)
if strcmpi(material, 'Copper'), data = [1.5,24,0.5; 2.5,32,0.8;
    4,42,1.2; 6,54,1.7; 10,73,2.8; 16,98,4.5; 25,129,7.0; 35,158,9.5;
    50,188,13.0; 70,232,18.0; 95,275,24.0; 120,315,30.0];
else, data = [16,76,3.0; 25,100,4.5; 35,123,6.0; 50,146,8.0;
    70,180,11.5; 95,215,15.0; 120,245,19.0; 150,280,23.0]; end
standard_sections_data = struct('section', num2cell(data(:,1)), '
    ampacity', num2cell(data(:,2)), 'cost_per_meter', num2cell(data(:,3))
    );
idx = find(data(:,1) >= s_required, 1, 'first');
if isempty(idx), selected_section = standard_sections_data(end);
else, selected_section = standard_sections_data(idx); end
end
```

Listing 21: Selects the next available standard conductor section from a table.

D.22 G4_calculateLifecycleCost.m

```
function [total_cost, cable_cost, loss_cost] = G4_calculateLifecycleCost
    (section_data, lineType, d, I, sigma, years, hours_per_year,
    cost_per_kWh)
s = section_data.section; cost_per_meter = section_data.cost_per_meter;
if strcmp(lineType, 'three-phase'), num_conductors = 3; else,
    num_conductors = 2; end
cable_cost = d * cost_per_meter * num_conductors;
total_resistance = (1/sigma) * d / s;
power_loss_watts = num_conductors * (I^2) * total_resistance;
total_kWh_lost = (power_loss_watts / 1000) * hours_per_year * years;
loss_cost = total_kWh_lost * cost_per_kWh;
total_cost = cable_cost + loss_cost;
end
```

Listing 22: Calculates the lifecycle cost (initial + losses) for a given section.

D.23 G5_plotEconomicAnalysis.m

```
function [] = G5_plotEconomicAnalysis(sections, cable_costs, loss_costs,
    total_costs, s_technical, s_optimal)
figure;
section_labels = arrayfun(@(x) num2str(x.section), sections, '
    UniformOutput', false);
x_categories = categorical(section_labels);
```

```
x_categories = reordercats(x_categories, section_labels);
bar_data = [cable_costs', loss_costs'];
bar(x_categories, bar_data, 'stacked');
hold on;
plot(x_categories, total_costs, 'd-r', 'LineWidth', 2, 'MarkerFaceColor', 'r', 'MarkerSize', 8, 'DisplayName', 'Total Lifecycle Cost');
optimal_idx = find([sections.section] == s_optimal);
if ~isempty(optimal_idx)
    text(x_categories(optimal_idx), total_costs(optimal_idx), '  Optimal', 'Color', 'red', 'FontWeight', 'bold');
end
technical_idx = find([sections.section] == s_technical);
if ~isempty(technical_idx)
     text(x_categories(technical_idx), total_costs(technical_idx), 'Technical Min.', 'Color', 'black', 'VerticalAlignment', 'top');
end
title('Economic Analysis of Conductor Sizing');
xlabel('Standard Conductor Section (mm^2)');
ylabel('Total Lifecycle Cost (EUR)');
legend({'Initial Cable Cost', 'Cost of Energy Losses', 'Total Lifecycle Cost'}, 'Location', 'northoutside', 'Orientation', 'horizontal');
grid on; hold off;
end
```

Listing 23: Plots the economic analysis results to find the optimal section.

D.24 G6_verifyFinalDesign.m

```
function final_vd_percent = G6_verifyFinalDesign(lineType, d, I, cos_phi, sigma, s_final, U_source)
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else, phase_factor = 2; end
deltaU_volts = (phase_factor * d * I * cos_phi) / (sigma * s_final);
final_vd_percent = (deltaU_volts / U_source) * 100;
end
```

Listing 24: Performs a final verification of the voltage drop for the chosen design.